

机器学习导论

第六讲：感知器网络与优化



人工智能系

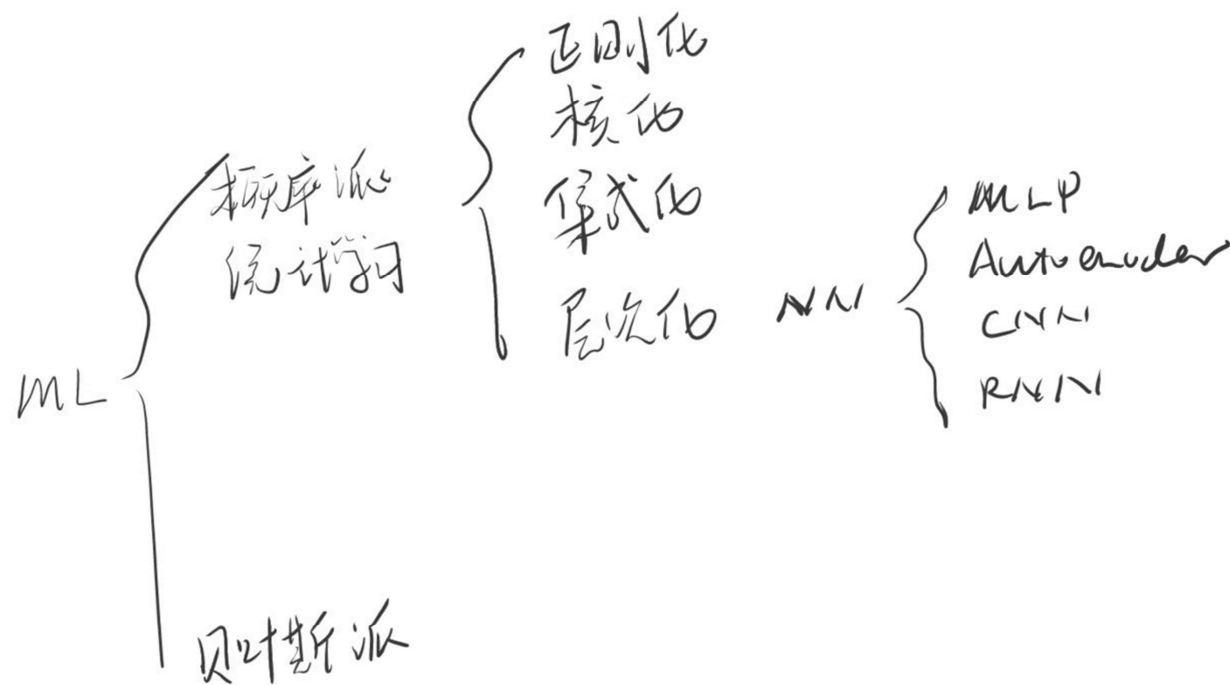
晁飞副教授

fchao@xmu.edu.cn



廈門大學
XIAMEN UNIVERSITY

机器学习流派

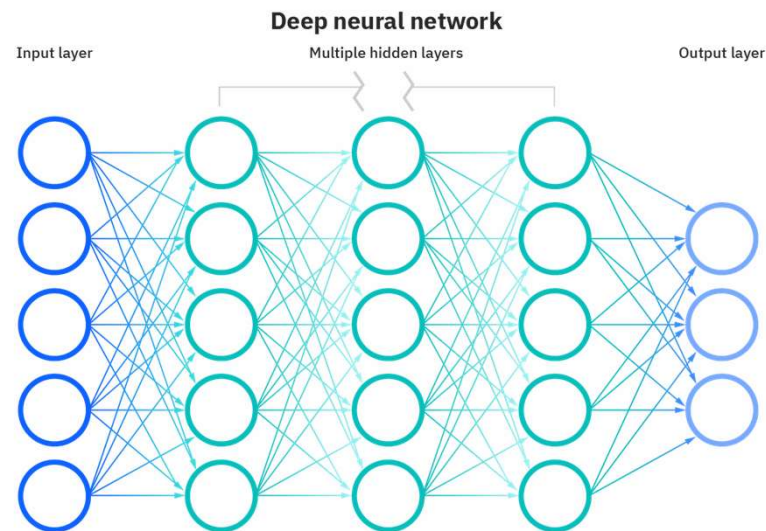


提纲

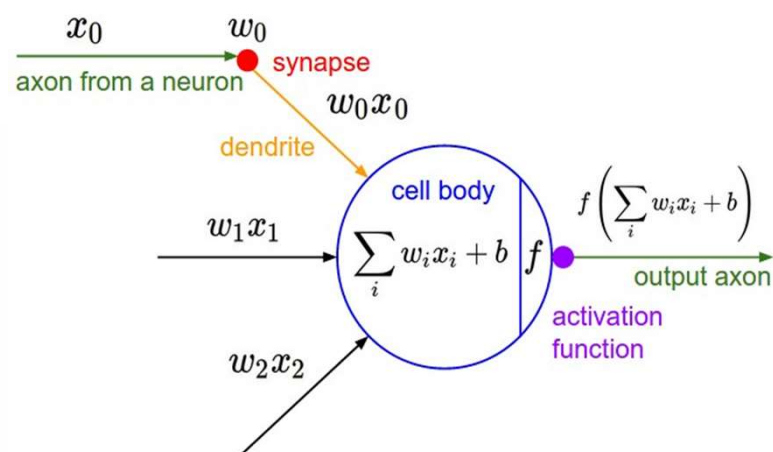
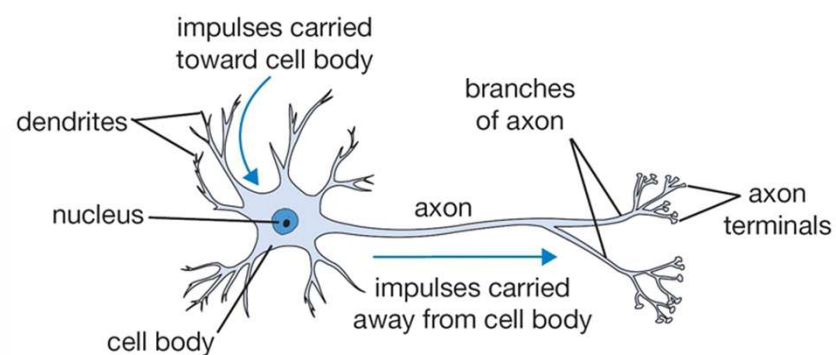
- 简介
- 激活函数
- 损失函数
- 参数优化

神经网络

- 神经网络，也称人工神经网络 (ANN)，是机器学习的子集，并且是深度学习算法的核心。其名称和结构是受人类大脑的启发，模仿了生物神经元信号相互传递的方式。
- 人工神经网络 (ANN) 由节点层组成，包含输入层、隐藏层和输出层。
- 每个节点称为一个人工神经元，它们连接到另一个节点，具有相关的权重。



人工神经元 (Neuron)

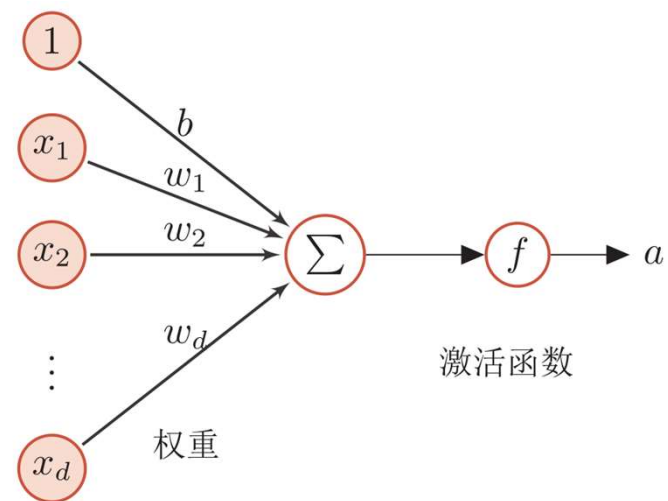


线性分类器

$$f(x) = \sum_{i=1}^d w_i x + b$$

□ 感知机 (Perception)

- $f(x) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b > 0 \\ -1 & \text{otherwise} \end{cases}$



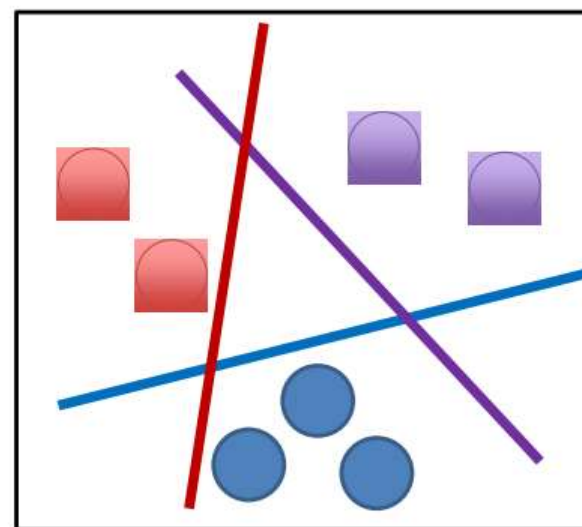
线性可分与线性不可分

□ 线性分类器：

$$y = W^T x + b$$

- 可以找到线性分界面能把不同类别的样本没有错误的分开
- 如果训练数据线性可分，感知器算法是可以停下来的
- 问：用感知器算法找到超平面是不是最优的？

线性可分



感知器算法

1. 随机选择 w^T 和 b
2. 取一个训练样本 (X, y)
 1. 如果 $w^T x + b > 0$ 且 $y = -1$, 则 $w = w - x$, $b = b - 1$
 2. 如果 $w^T x + b < 0$ 且 $y = +1$, 则 $w = w + x$, $b = b + 1$
3. 再去另一个 (X, y) , 会到第2步
4. **终止条件:** 直到所有输入输出对都不满足2.1和2.2任意一个条件, 退出

如果与上述条件相反:

$$\begin{aligned} & W(\text{新})^T X + b(\text{新}) \\ &= (W(\text{旧}) - X)^T X + b(\text{旧}) - 1 \\ &= [(W(\text{旧})^T X + b(\text{旧})] - (X^T X + 1) \\ &= [(W(\text{旧})^T X + b(\text{旧})] - (\|X\|^2 + 1) \\ &\leq [(W(\text{旧})^T X + b(\text{旧})] - 1 \end{aligned}$$

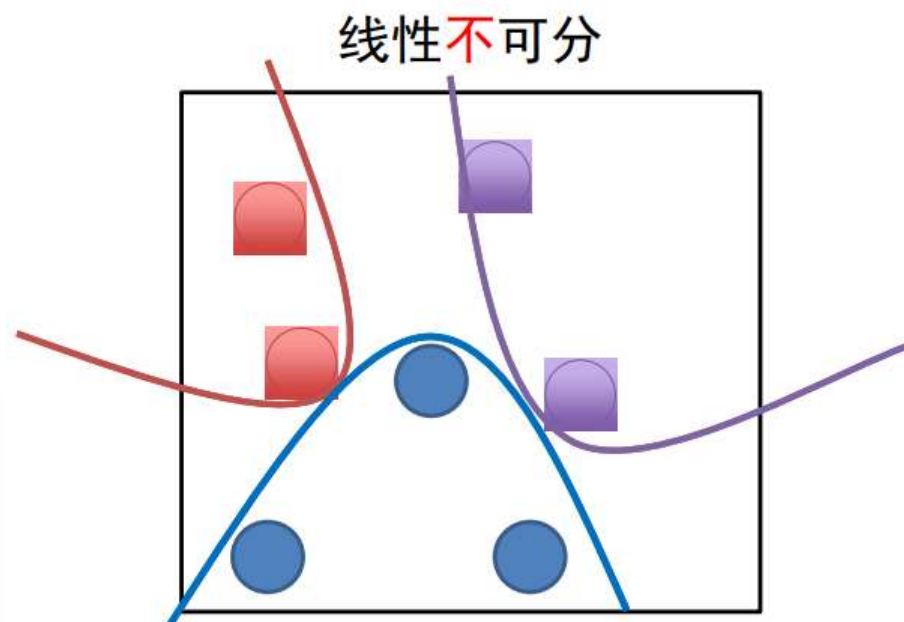
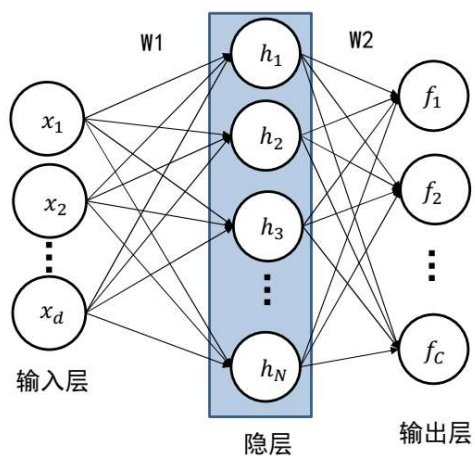
$$\begin{aligned} & W(\text{新})^T X + b(\text{新}) \\ &= (W(\text{旧}) + X)^T X + b(\text{旧}) + 1 \\ &= [(W(\text{旧})^T X + b(\text{旧})] + (X^T X + 1) \\ &= [(W(\text{旧})^T X + b(\text{旧})] + (\|X\|^2 + 1) \\ &\geq [(W(\text{旧})^T X + b(\text{旧})] + 1 \end{aligned}$$

线性可分与线性不可分

□ 两层全连接层的神经网络：

- $h = W_1^T x + b_1$

- $y = W_2^T h + b_2$



□ 定理：如果分线性函数采用阶跃函数，那么三层神经网络可以模拟任意的非线性函数

多分类情况

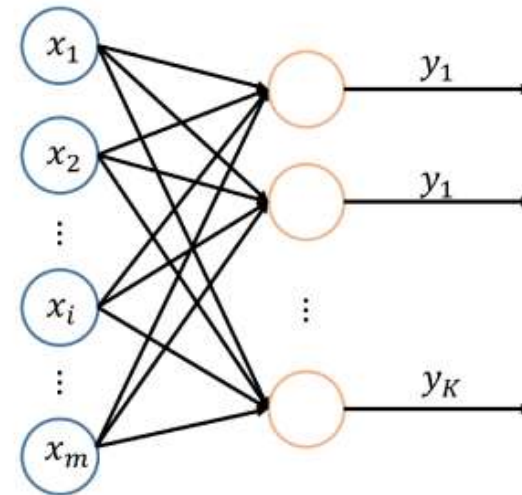
- 多个输出神经元，每一个对应一个类别：

$$y_i = \mathbf{w}_i^T \mathbf{x} + b_i$$

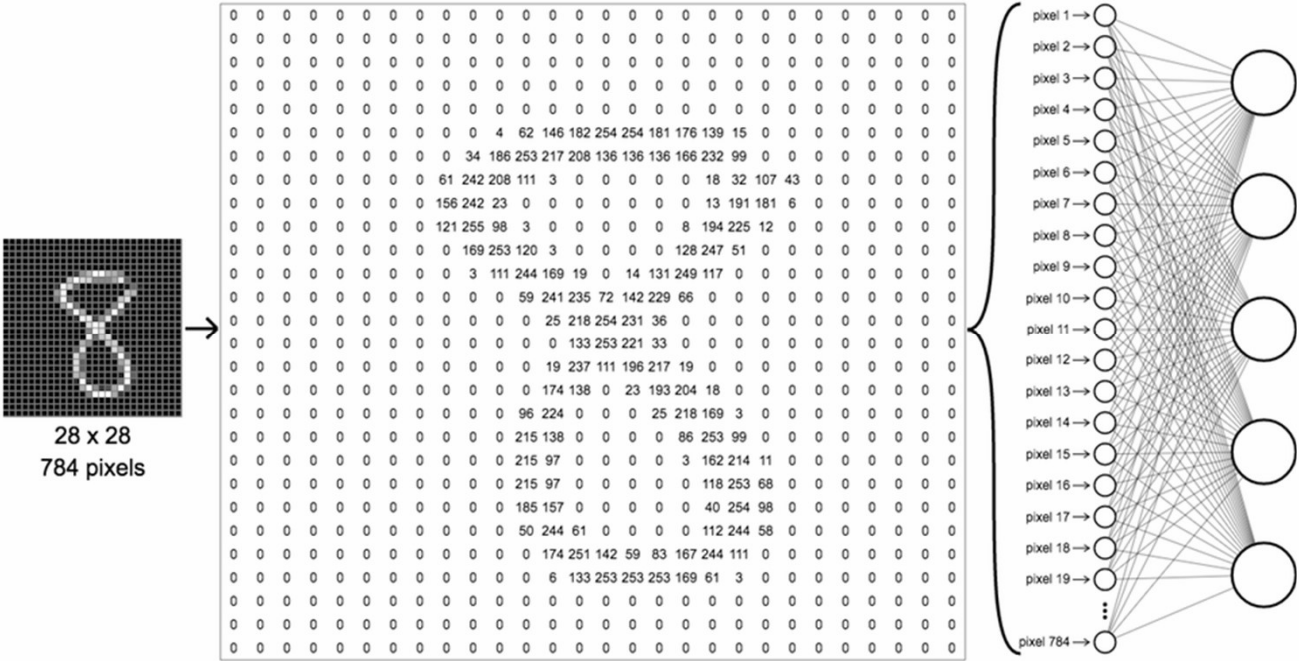
- 矩阵的形式：

$$\mathbf{y} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$$

$$\mathbf{W} \in R^{M \times K}$$



手写数字识别网络



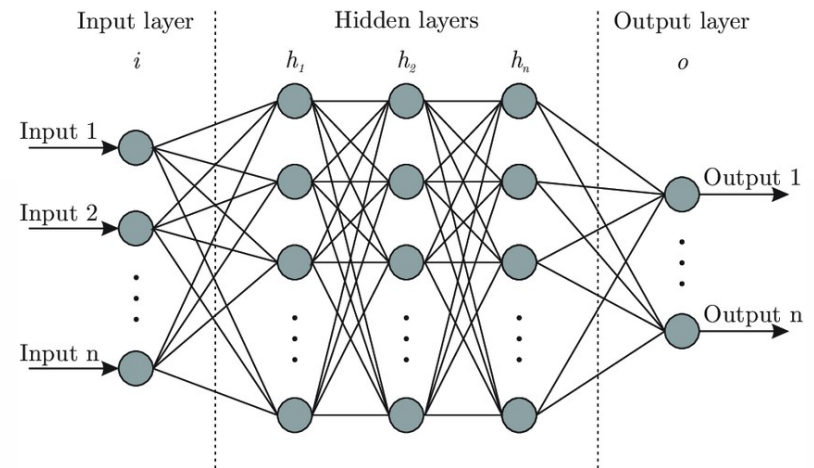
多层神经网络

□ 中间有一个或者多个**隐含层**

□ 每一层对应的参数 W_i 和 b_i

□ 隐含层的输出:

$$h_i = f(W_i^T h_{i-1} + b_i)$$



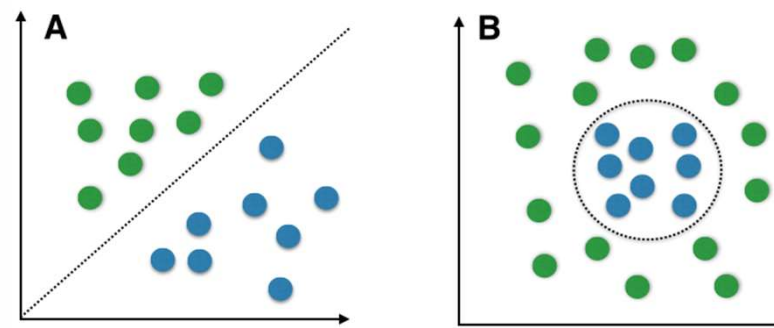
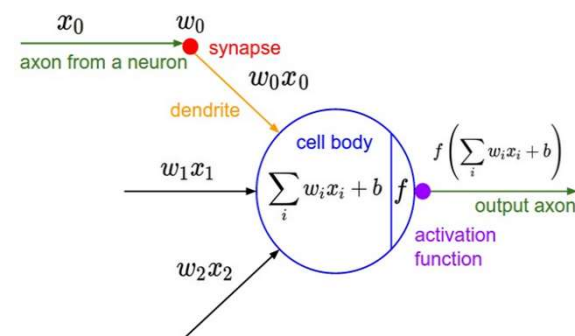
提纲

- 简介
- 激活函数
- 损失函数
- 参数优化

常用的激活函数

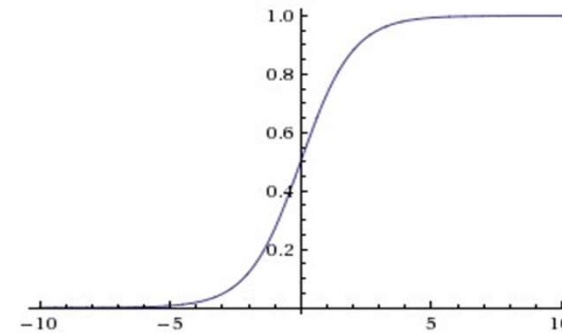
□ 通过对每一个神经元采用非线性的激活函数，让神经网络可以表示非线性

- Sigmoid
- Tanh
- ReLU
- ...



1. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



为什么一定是这个样子？

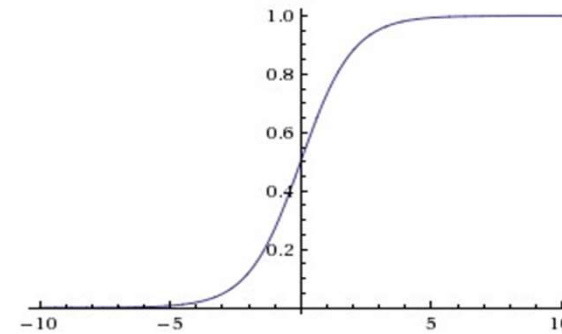
取一个实值并将其压缩为 [0,1]

缺点：

- 饱和和终止梯度
- 不以零为中心
- 计算成本高

1. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



梯度:

$$\frac{d\sigma}{dx} = \frac{-e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} * \frac{-e^{-x}}{1 + e^{-x}} = \sigma * (1 - \sigma)$$

取一个实值并将其压缩为 [0,1]

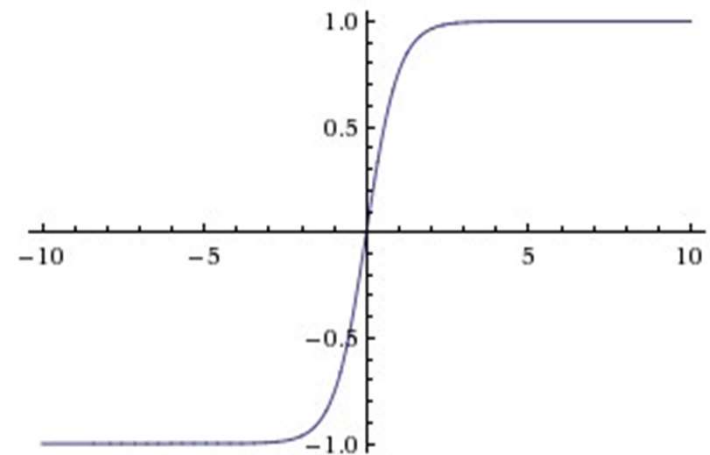
缺点:

- 饱和和终止梯度
- 不以零为中心
- 计算成本高

2. Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

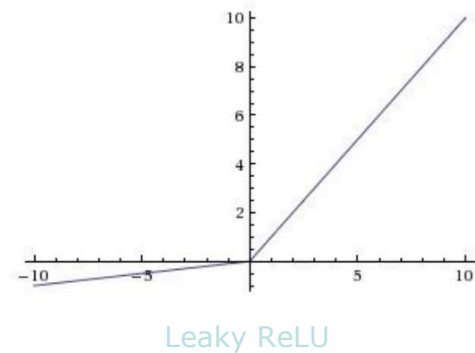
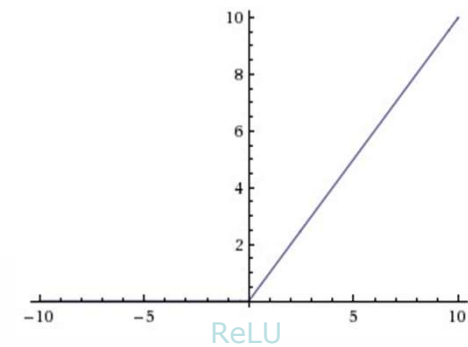
- 将实数值压缩为 $[-1, 1]$
- 零点为中心
- 梯度没了!!!



3. ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

- 计算简单
- 加快收敛



常见激活函数及其导数

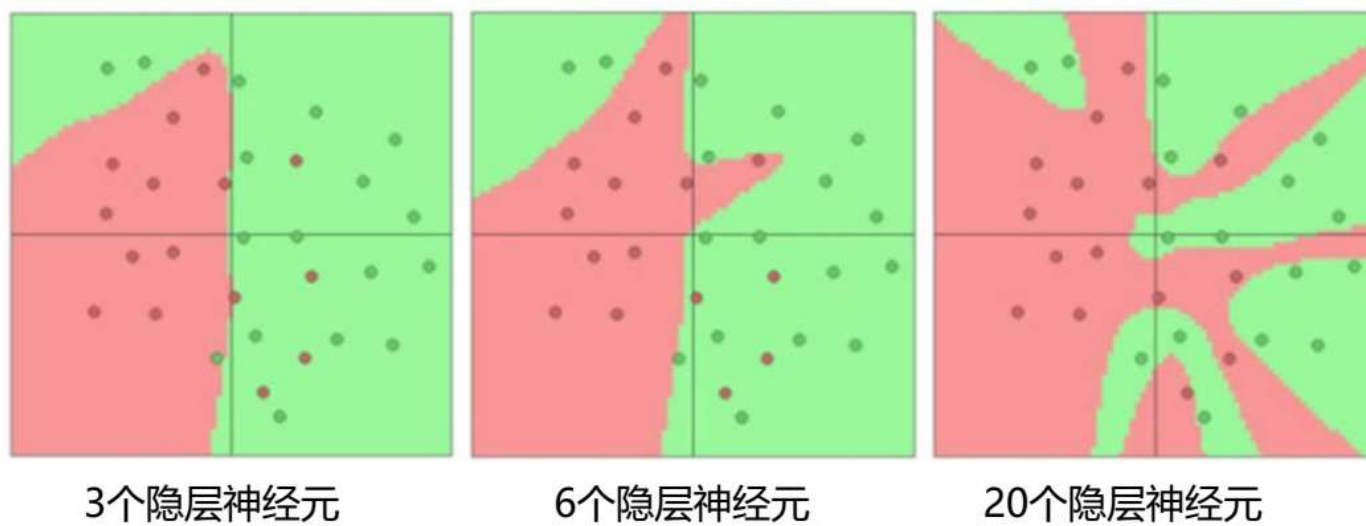
激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

网络结构设计

1. 用不用隐层，用一个还是用几个隐层？（深度设计）
2. 每个隐层设置多少个神经元比较合适？（宽度设计）

没有统一的答案！

网络结构设计



结论：神经元个数越多，分界面就可以越复杂，在这个集合上的分类能力就越强。

网络结构设计

- 依据分类任务的难易程度来调整神经网络模型的复杂程度。
。分类任务越难，设计的神经网络结构就应该越深、越宽。
。但是，需要注意的是对训练集分类精度最高的全连接神经网络模型，在真实场景下识别性能未必是最好的（过拟合）。

小结

- 全连接神经网络组成：一个输入层、一个输出层及多个隐层；
- 输入层与输出层的神经元个数由任务决定，而隐层数量以及每个隐层的神经元个数需要人为指定；
- 激活函数是全连接神经网络中的一个重要部分，缺少了激活函数，全连接神经网络将退化为线性分类器。

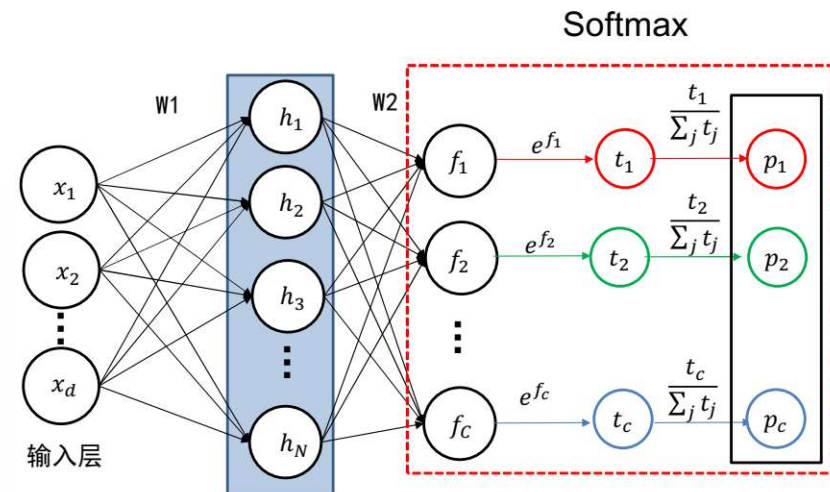
提纲

- 简介
- 激活函数
- 损失函数
- 参数优化

Softmax函数

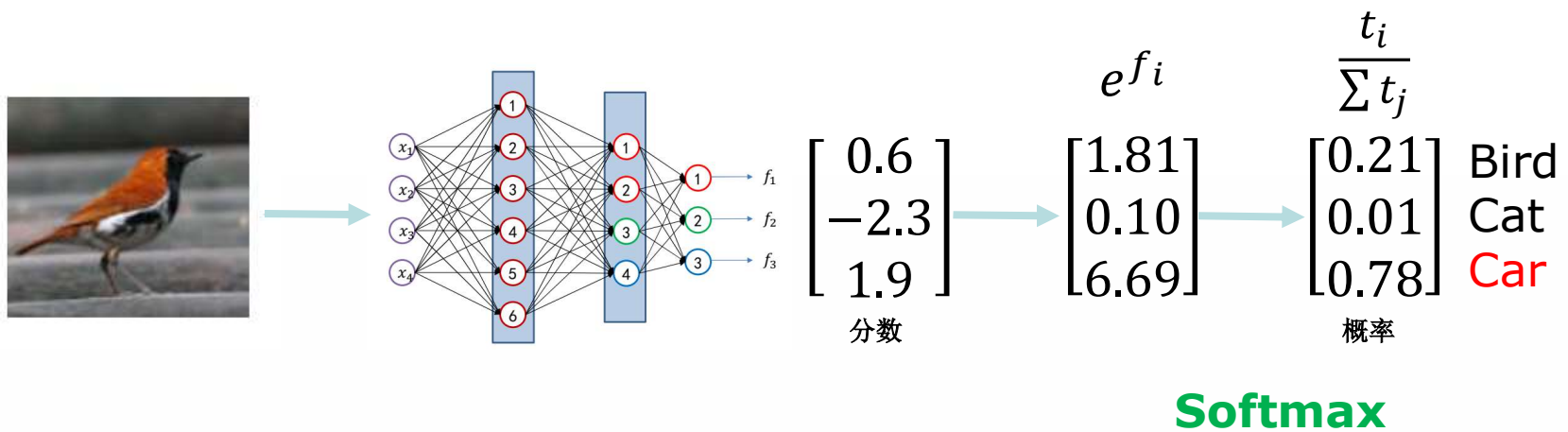
$$\text{Softmax}(\mathbf{x}) = \frac{1}{\sum_{j=1}^K e^{x_j}} \begin{bmatrix} e^{x_1} \\ e^{x_2} \\ \vdots \\ e^{x_K} \end{bmatrix}$$

□ 将输出转换成归一化之后的概率形式。



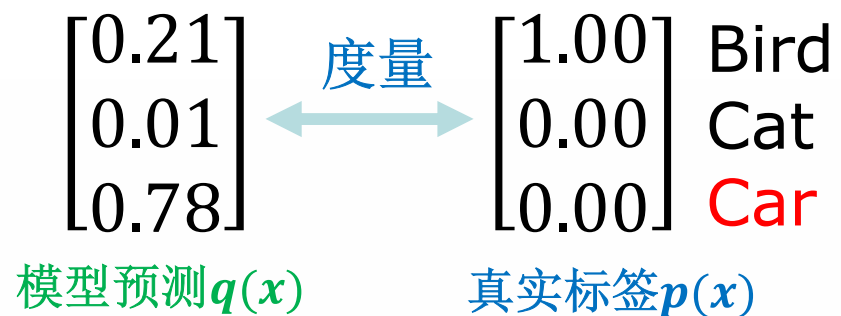
Softmax函数

□ 示例:



交叉熵损失函数 (Cross-Entropy)

□ 如何度量模型的分类输出与真实标签之间的距离？



交叉熵损失函数 (Cross-Entropy)

□ 熵: $H(p) = -\sum_{i=1}^C p_i(x) \log(p_i(x))$

□ 交叉熵: $H(p, q) = -\sum_{i=1}^C p_i(x) \log(q_i(x))$

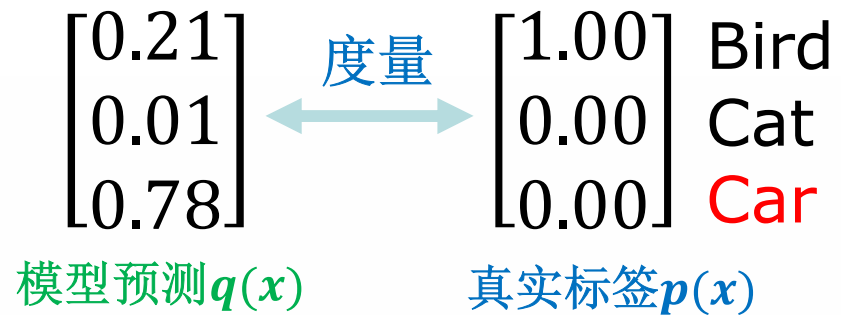
□ 相对熵: $KL(p||q) = -\sum_{i=1}^C p_i(x) \log\left(\frac{q_i(x)}{p_i(x)}\right)$

相对熵也叫KL散度(KL divergence);
(用来度量两个分布之间的不相似性)

$$H(p, q) = H(p) + KL(p||q)$$

交叉熵损失函数 (Cross-Entropy)

□ 如何度量模型的分类输出与真实标签之间的距离？



交叉熵: $H(p, q) = -\sum_{i=1}^C p_i(x) \log(q_i(x))$

交叉熵损失函数 (Cross-Entropy)

□ 如何度量模型的分类输出与真实标签之间的距离？



$\begin{bmatrix} 0.21 \\ 0.01 \\ 0.78 \end{bmatrix}$
模型预测 $q(x)$

度量

$\begin{bmatrix} 1.00 \\ 0.00 \\ 0.00 \end{bmatrix}$ Bird
Cat
Car
真实标签 $p(x)$

$$L_{CE} = -\log(0.21) = 1.56$$

$$\text{交叉熵损失: } L_{CE} = -\sum_{i=1}^C p_i(x) \log(q_i(x))$$

交叉熵损失函数 (Cross-Entropy)

分数	概率	交叉熵损失
[10, -2, 3]	[0.999082816, 6.13858E-06, 0.000911046]	0.0004
[10, 9, 9]	[0.576116885, 0.211941558, 0.211941558]	0.2395
[10, -100, -100]	[1, 1.68891E-48, 1.68891E-48]	1.5×10^{-48}

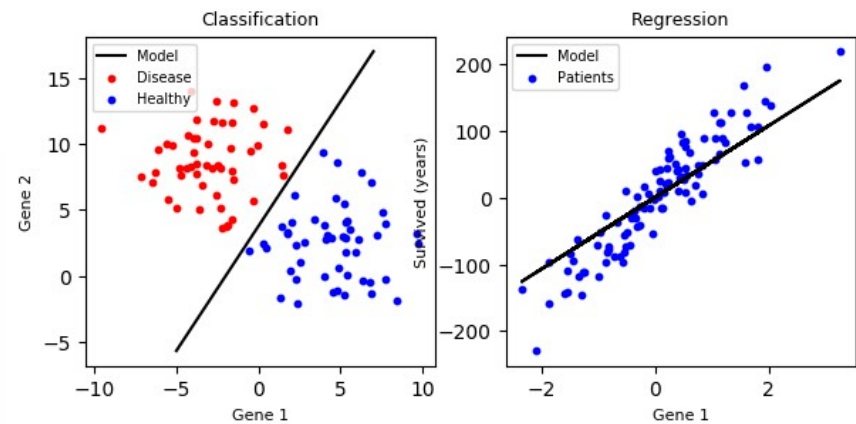
回归任务 (Regression)

□ L1-Loss

$$L_1 = \sum_{i=1}^n |y_{true} - y_{predicted}|$$

□ L2-Loss (MSE)

$$L_2 = \sum_{i=1}^n (y_{true} - y_{predicted})^2$$

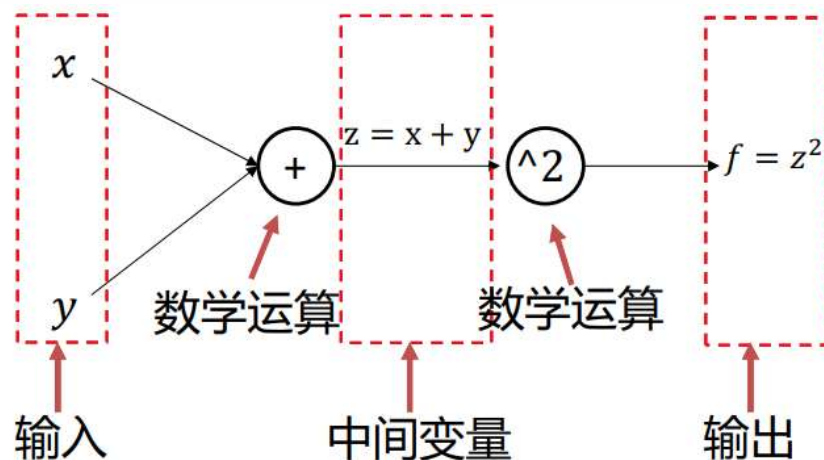


提纲

- 简介
- 激活函数
- 损失函数
- 参数优化

计算图与反向传播算法

- 计算图是一种有向图，用来表达输入、输出以及中间变量之间的计算关系，图中的每一个节点对应着一种数学运算。
- 函数 $f = (x + y)^2$ 的计算图

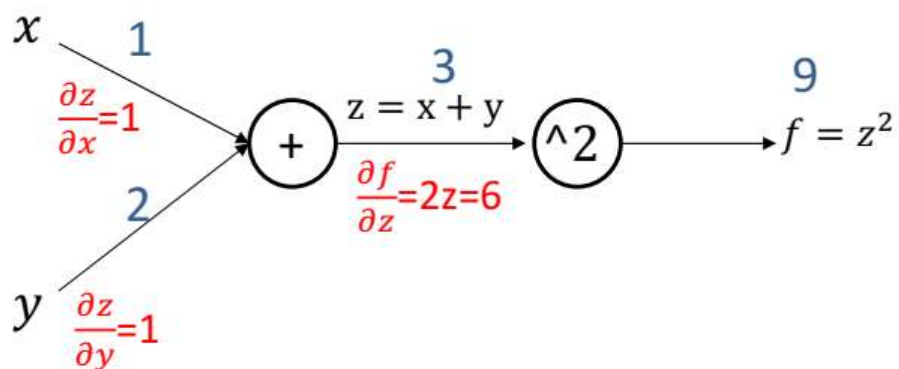


计算图的前、反向计算

□ 函数 $f = (x + y)^2$ 的计算图

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial z} \frac{\partial z}{\partial x}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z} \frac{\partial z}{\partial y}$$



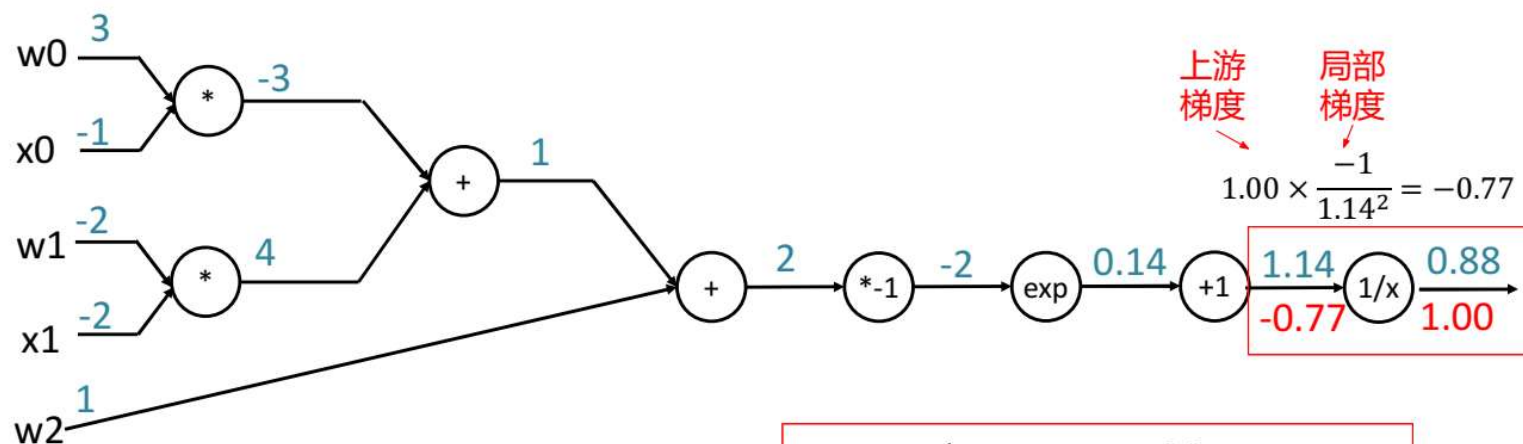
计算图小节

- 任意复杂的函数，都可以用计算图的形式表示
- 在整个计算图中，每个单元都会得到一些输入，然后，进行下面两个计算：
 - a) 这个单元的输出值
 - b) 其输出值关于输入值的局部梯度。
- 利用链式法则，每个单元应该将回传的梯度乘以它对其的输入的局部梯度，从而得到整个网络的输出对该单元的每个输入值的梯度。

反向传播示例2

$$\frac{\partial f}{\partial w_1} = \frac{\partial f}{\partial u} \cdot \frac{\partial u}{\partial w_1} = \frac{\partial f}{\partial u} \cdot z_1$$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

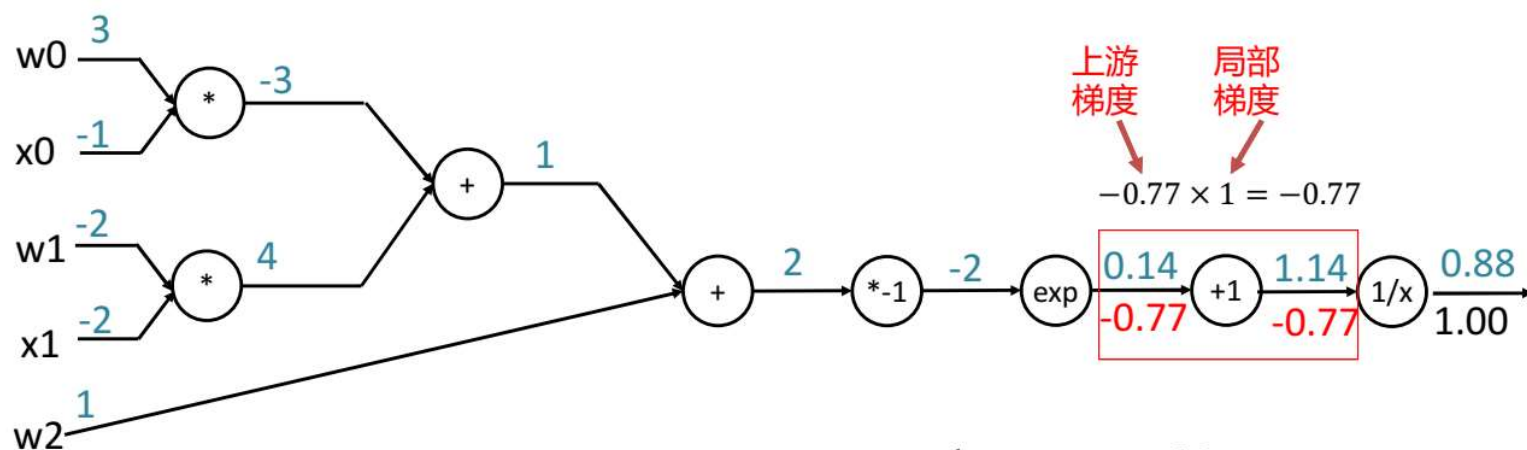
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

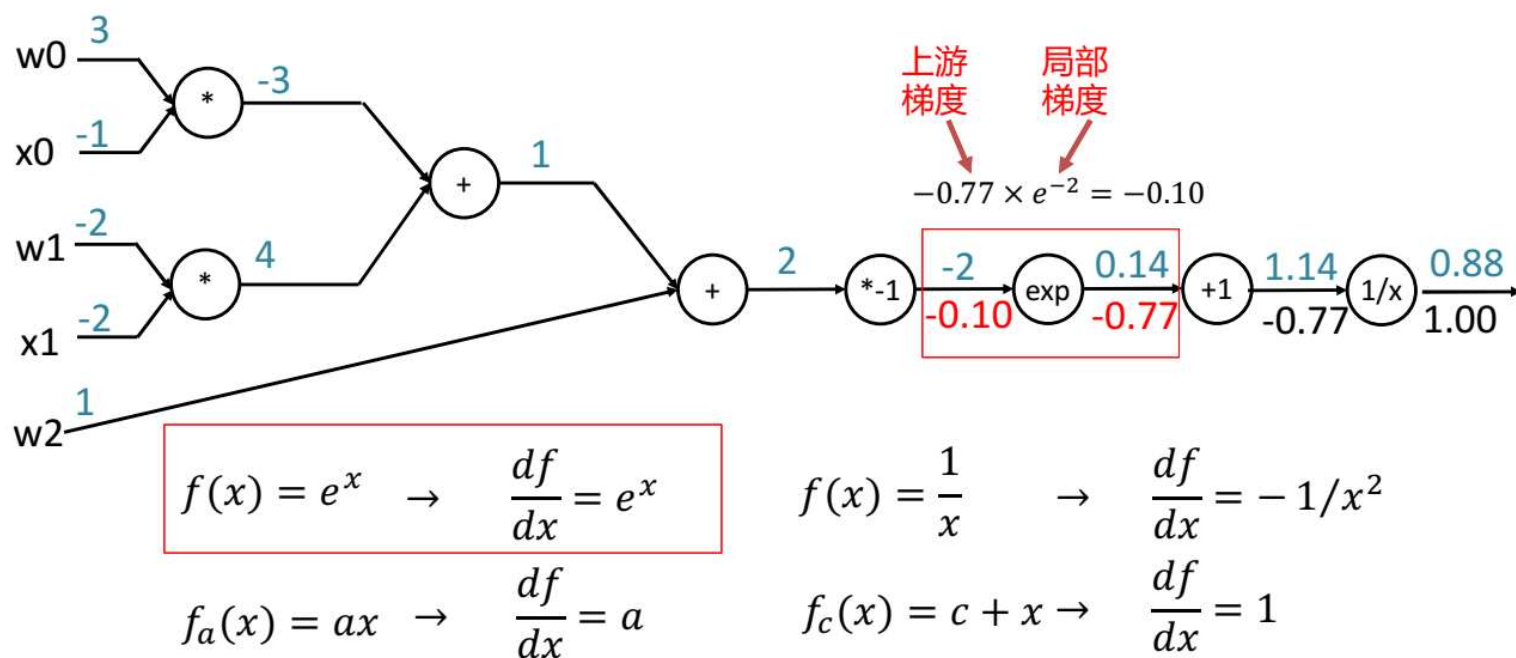
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

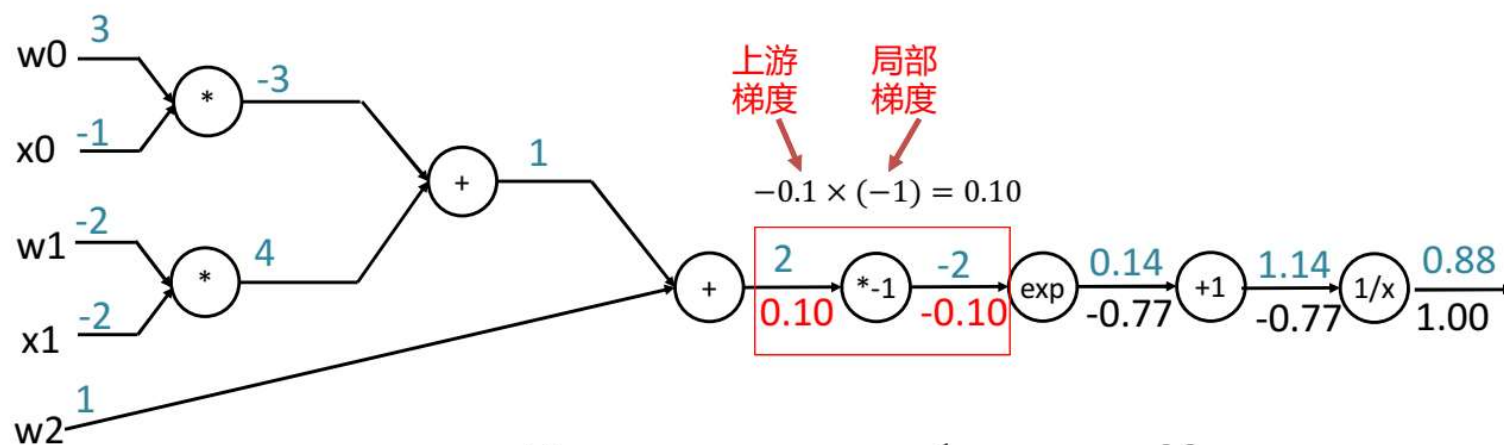
反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

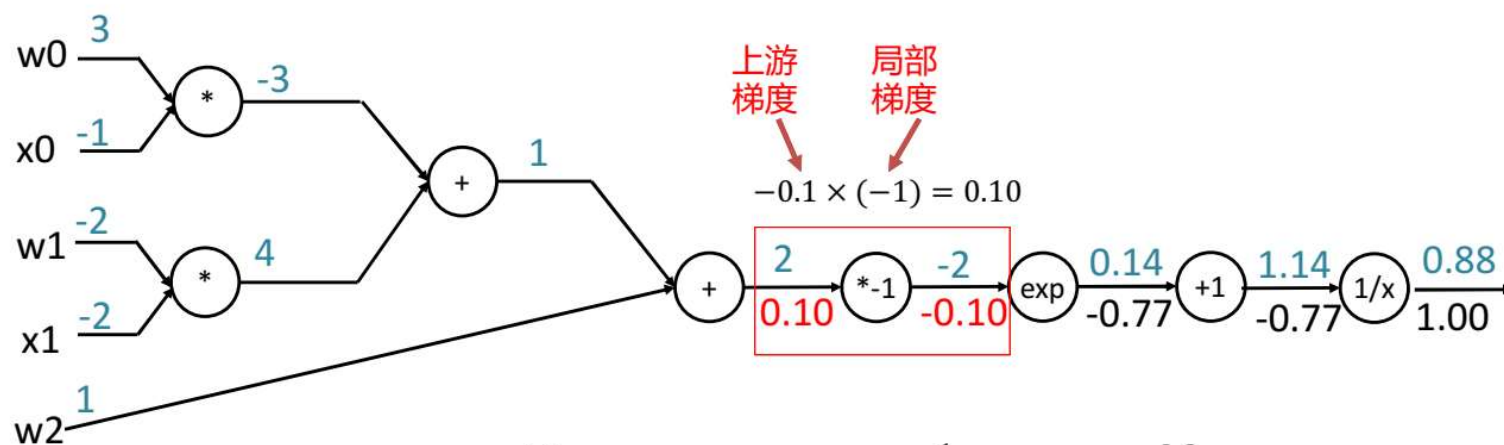
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

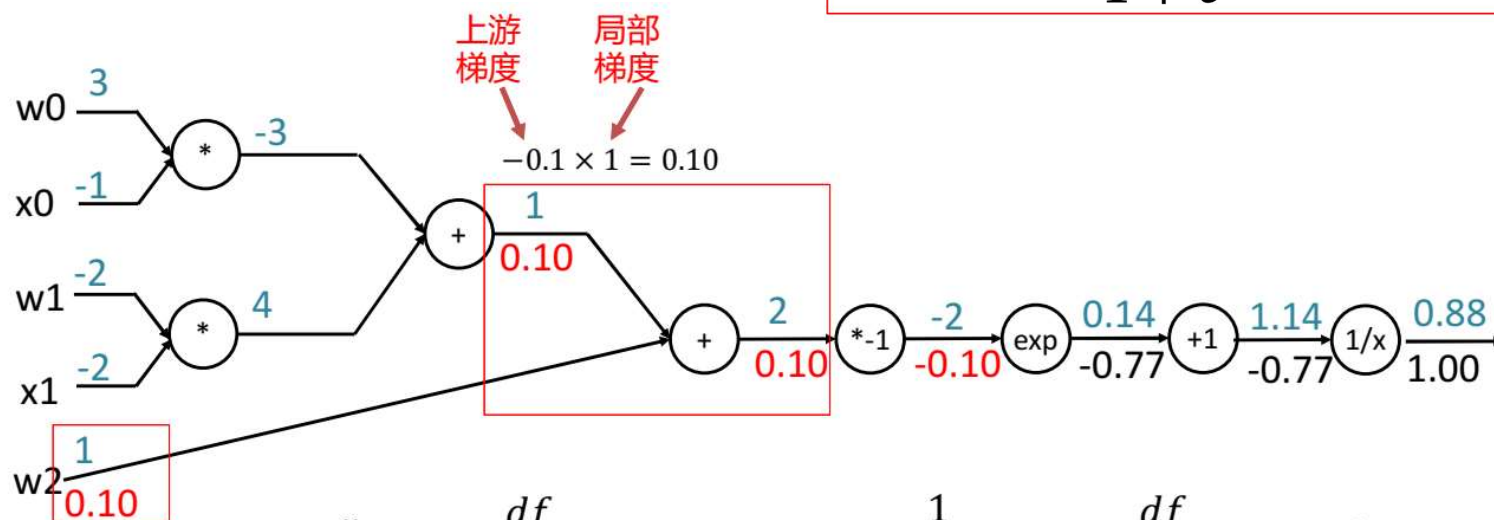
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

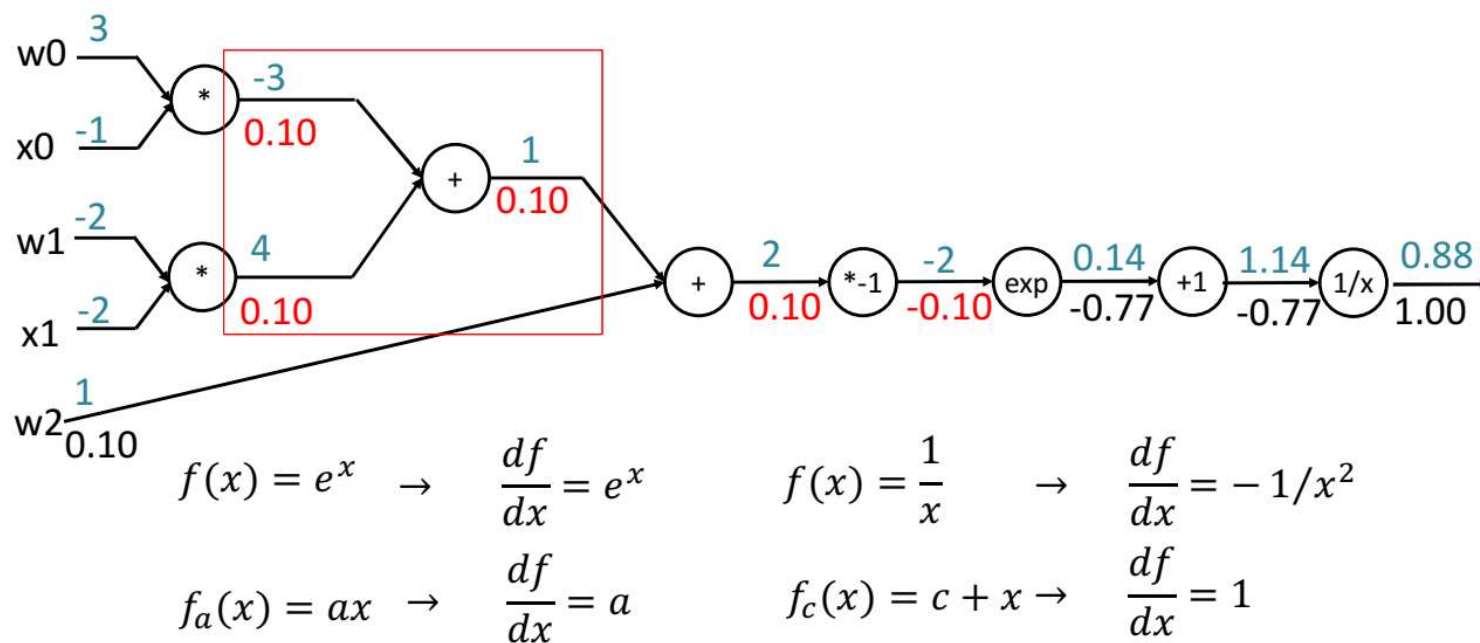
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

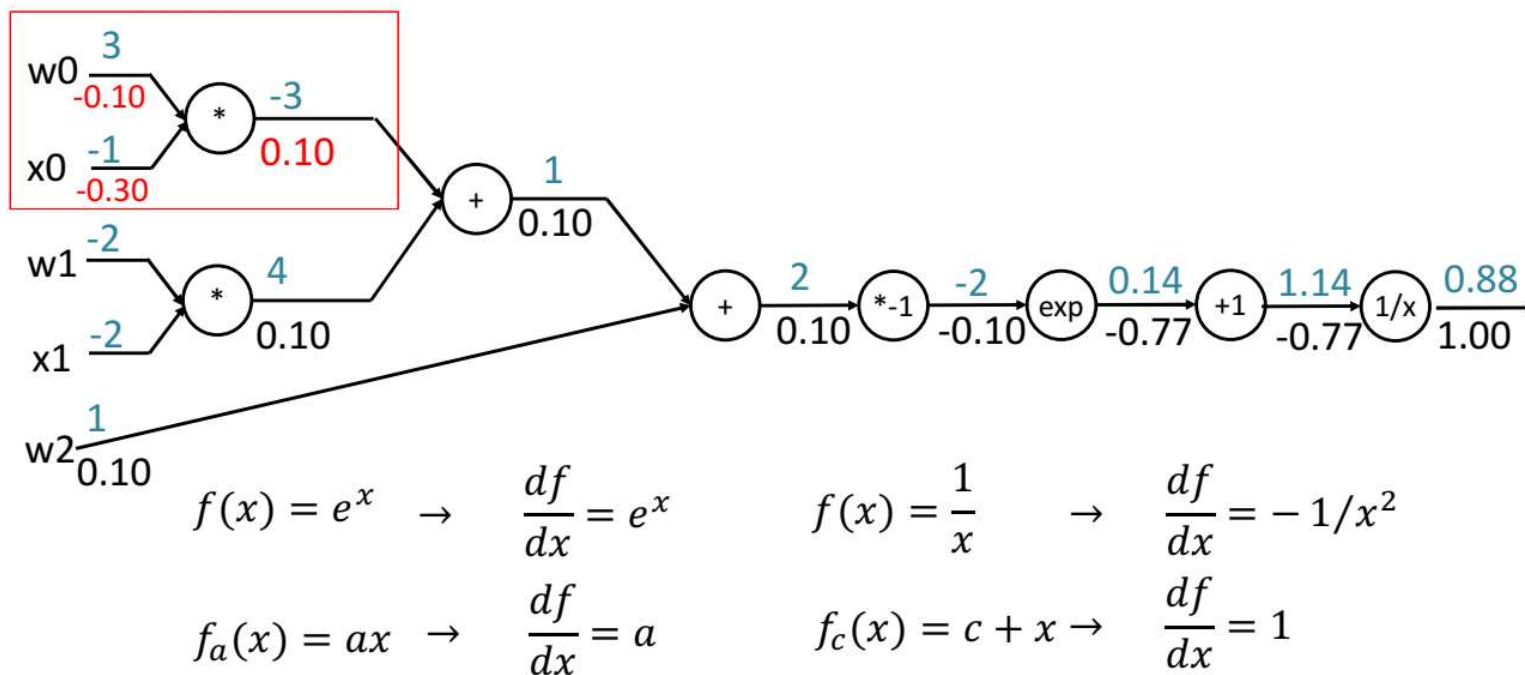
反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



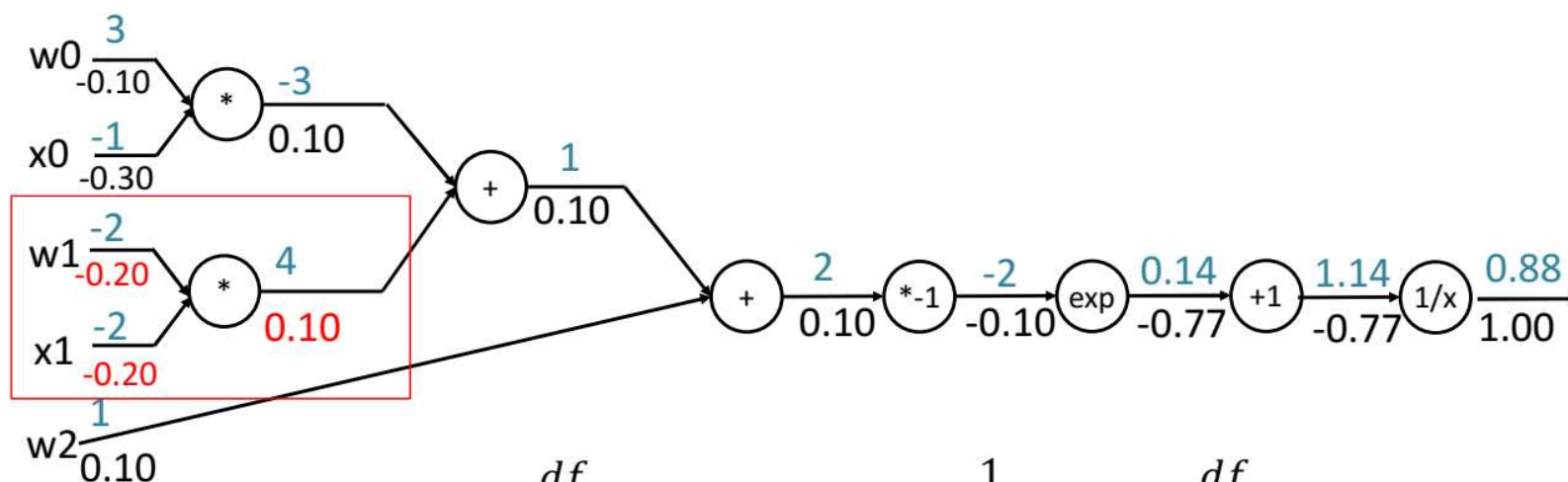
反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

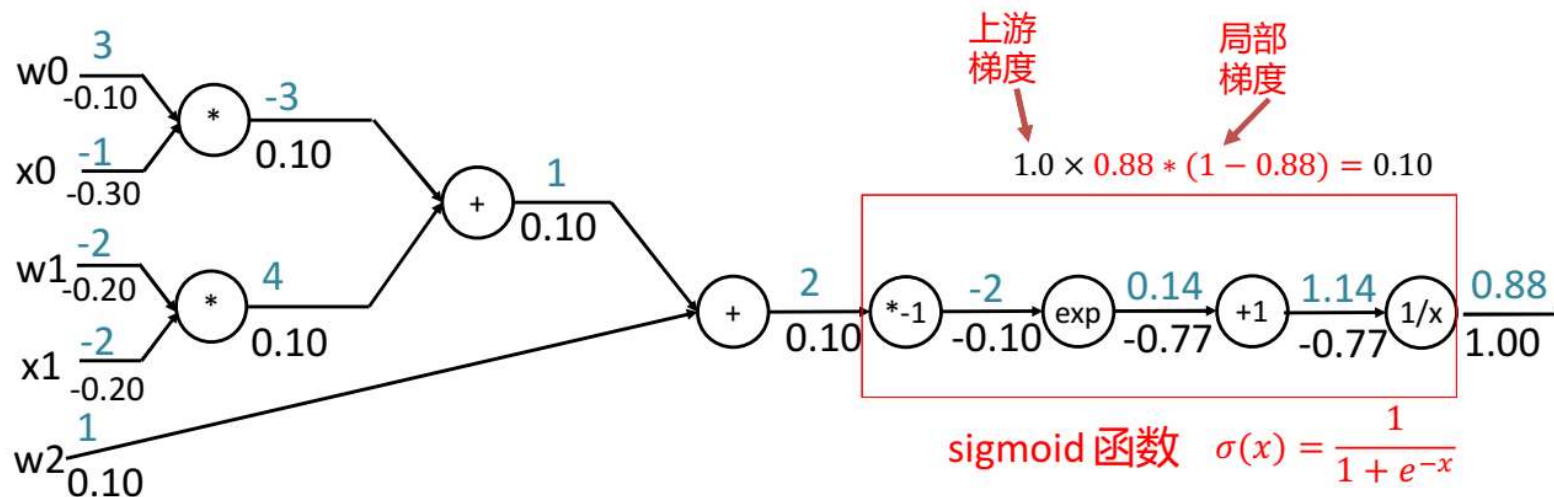
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

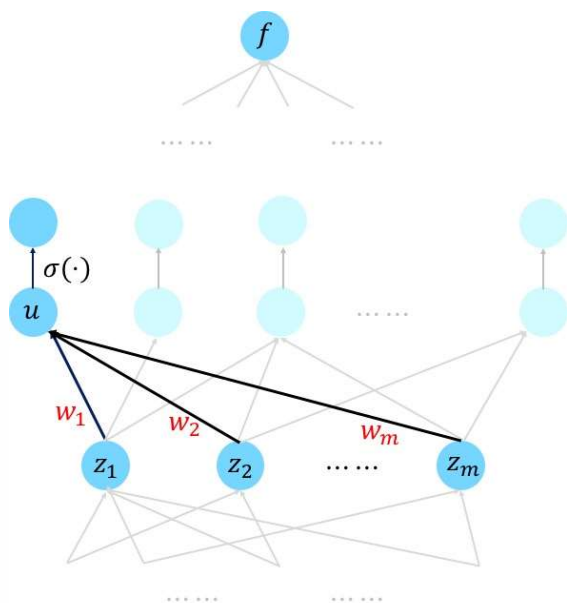
反向传播示例2

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

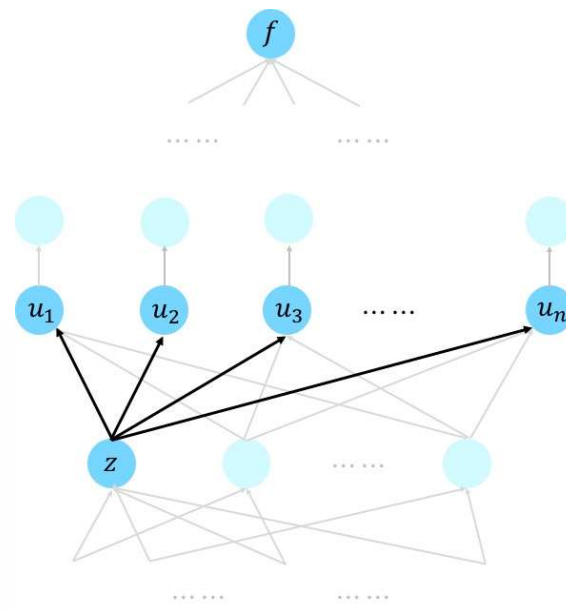


$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

反传传播算法



$$\frac{\partial f}{\partial w_1} = \frac{\partial f}{\partial u} \cdot \frac{\partial u}{\partial w_1} = \frac{\partial f}{\partial u} \cdot z_1$$



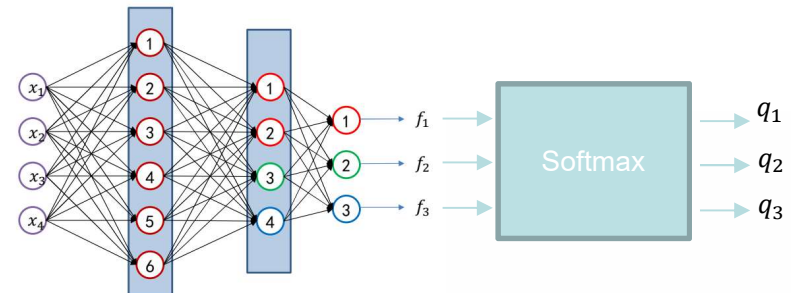
$$\frac{\partial f}{\partial z} = \sum_{j=1}^n \frac{\partial f}{\partial u_j} \cdot \frac{\partial u_j}{\partial z}$$

交叉熵损失函数的梯度

□ $L_{ce} = -\sum_{i=1}^C p_i(x) \log(q_i(x))$

□ $\frac{\partial L_{ce}}{\partial q_i} = -\frac{p_i}{q_i}$

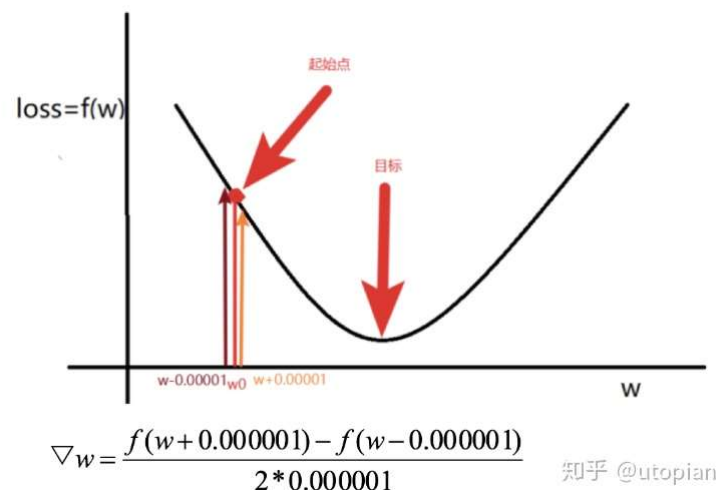
□ 后续再基于反向传播计算相应的梯度



梯度下降法

- 在最小化损失函数时，可以通过梯度下降法来一步步的迭代求解，得到最小化的损失函数，和模型参数值。

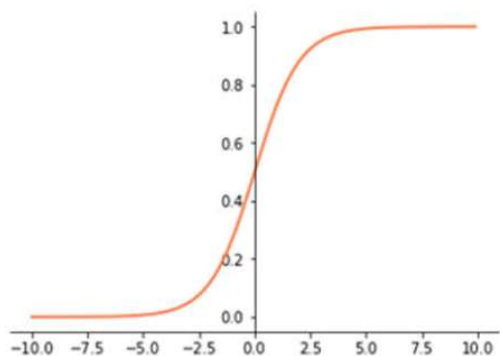
$$x = x - lr * \Delta x$$



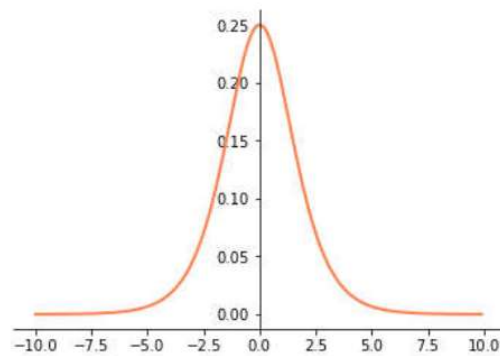
$$\frac{df(x)}{dx} = \frac{f(x+h) - f(x-h)}{2h}$$

激活函数的梯度

Sigmoid激活函数: $\sigma(x) = 1/(1 + e^{-x})$



sigmoid函数

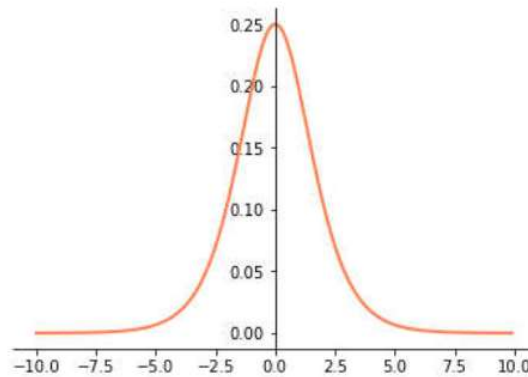


Sigmoid函数的导数函数

当输入值大于10或者小于-10时局部梯度都是0；非常不利于网络的梯度流传递的。

梯度消失

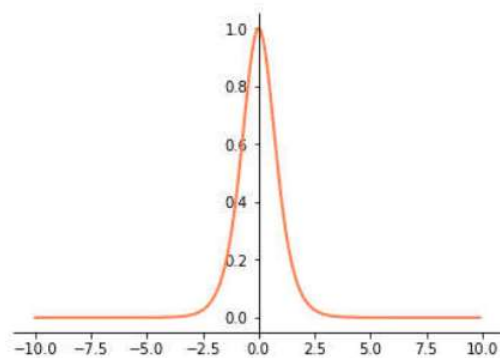
- 梯度消失是神经网络训练中非常致命的一个问题，其本质是由于链式法则的乘法特性导致的。



Sigmoid函数的导数函数

激活函数的梯度

双曲正切激活函数: $\tanh(x) = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



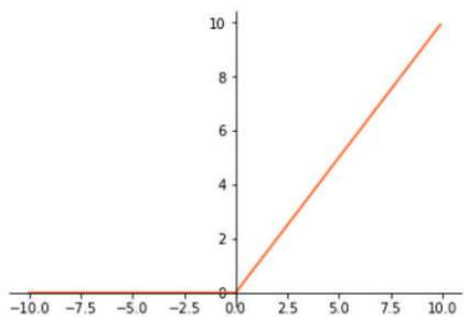
与sigmoid类似，局部梯度特性
不利于网络梯度流的反向传递

tanh函数

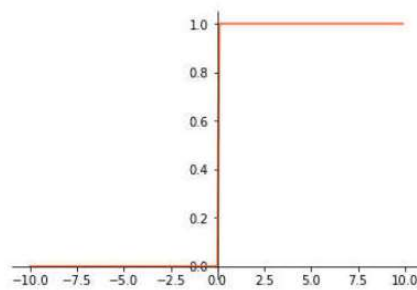
tanh函数的导数函数

激活函数的梯度

ReLU激活函数: $f(x) = \max(0, x)$



ReLU函数

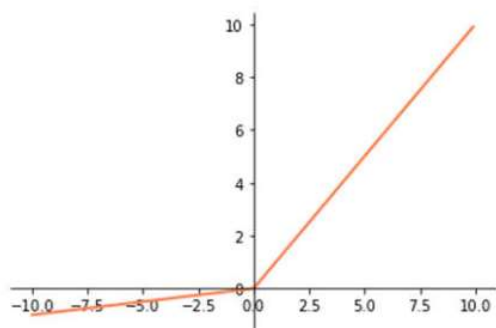


ReLU函数的导数函数

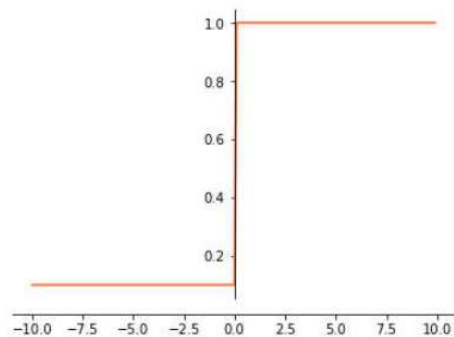
当输入大于0时，局部梯度永远不会为0，比较有利于梯度流的传递。

激活函数的梯度

Leaky ReLU激活函数: $f(x) = \max(0.01x, x)$



ReLU函数

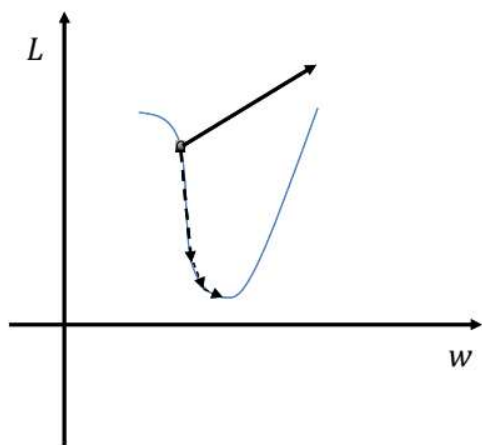


ReLU函数的导数函数

基本没有“死区”，也就是梯度永远不会为0。之所以说“基本”，是因为函数在0处没有导数。

梯度爆炸

□ 梯度爆炸也是由于链式法则的乘法特性导致的



梯度爆炸：断崖处梯度乘以学习率后会是一个非常大得值，从而“飞”出了合理区域，最终导致算法不收敛；

解决方案：把沿梯度方向前进的步长限制在某个值内就可以避免“飞”出了，这个方法也称为梯度裁剪。

采用随机梯度 (SGD) 下降的要点

□ 不用没输入一个样本就去更新

- Batch; Mini Batch

□ 对于每一个Epoch，需要随机打乱所有训练样本的次序

- 10,000 样本, Batch Size = 100, Batch 个数?

小结

- 利用反向传播算法计算网络每一个节点与参数的梯度
- 基于梯度下降算法进行参数优化
- 梯度消失与爆炸